

Pyspark DataFrame Transformations Scenarios



Calculating sales totals with groupBy() and sum()

Cmd 9

```
1 sales_data = [  
2     {"product_id": 1, "product_name": "Laptop", "price": 1200, "quantity": 2},  
3     {"product_id": 2, "product_name": "Smartphone", "price": 800, "quantity": 1},  
4     {"product_id": 1, "product_name": "Laptop", "price": 1200, "quantity": 1},  
5 ]  
6 df = spark.createDataFrame(sales_data)  
7  
8 sales_totals = df.groupBy("product_name").agg(sum("price").alias("total_price"), sum("quantity").alias("total_quantity"))  
9 print(sales_totals.display())
```

- ▶ (2) Spark Jobs
- ▶  df: pyspark.sql.dataframe.DataFrame = [price: long, product_id: long ... 2 more fields]
- ▶  sales_totals: pyspark.sql.dataframe.DataFrame = [product_name: string, total_price: long ... 1 more field]

Table  +

	product_name 	total_price 	total_quantity 	
1	Laptop	2400	3	
2	Smartphone	800	1	

 2 rows | 3.20 seconds runtime

Identifying top-selling products with sort() and limit()

Cmd 11



```
1 sales_data = [
2     {"product_id": 1, "product_name": "Laptop", "price": 1200, "quantity": 2},
3     {"product_id": 2, "product_name": "Smartphone", "price": 800, "quantity": 1},
4     {"product_id": 1, "product_name": "Laptop", "price": 1200, "quantity": 1},
5 ]
6 df = spark.createDataFrame(sales_data)
7
8 top_selling_products = df.groupBy("product_name").agg(sum("quantity").alias("total_quantity")).sort("total_quantity", ascending=False).limit(3)
9 print(top_selling_products.display())
```

► (2) Spark Jobs

►  df: pyspark.sql.dataframe.DataFrame = [price: long, product_id: long ... 2 more fields]

►  top_selling_products: pyspark.sql.dataframe.DataFrame = [product_name: string, total_quantity: long]

Table ▾ +

	product_name ▲	total_quantity ▲
1	Laptop	3
2	Smartphone	1

Analyzing customer purchase patterns with join() and groupBy()

Cmd 13

```
1 from pyspark.sql.functions import *
2 customer_data = [
3     {"customer_id": 1, "name": "Raveendra", "email": "Raveendra@gmail.com"},
4     {"customer_id": 2, "name": "Reshwanth", "email": "Reshwanth@gmail.com"},
5 ]
6 sales_data = [
7     {"order_id": 1, "customer_id": 1, "product_id": 1, "quantity": 1},
8     {"order_id": 2, "customer_id": 2, "product_id": 2, "quantity": 2},
9 ]
10
11 customer_df = spark.createDataFrame(customer_data)
12 sales_df = spark.createDataFrame(sales_data)
13
14 joined_df = customer_df.join(sales_df, on="customer_id")
15 purchase_patterns = joined_df.groupBy("name").agg(count("order_id"))
16 print(purchase_patterns.display())
```

► (4) Spark Jobs

-  customer_df: pyspark.sql.dataframe.DataFrame = [customer_id: long, email: string ... 1 more field]
-  sales_df: pyspark.sql.dataframe.DataFrame = [customer_id: long, order_id: long ... 2 more fields]
-  joined_df: pyspark.sql.dataframe.DataFrame = [customer_id: long, email: string ... 4 more fields]
-  purchase_patterns: pyspark.sql.dataframe.DataFrame = [name: string, count(order_id): long]

Table ▾ +

	name ▲	count(order_id) ▲
1	Raveendra	1
2	Reshwanth	1

Calculating customer lifetime value (CLV) with groupBy() and sum()

Cmd 15

```
1 order_data = [
2     {"order_id": 1, "customer_id": 1, "order_amount": 100},
3     {"order_id": 2, "customer_id": 1, "order_amount": 50},
4     {"order_id": 3, "customer_id": 2, "order_amount": 200},
5 ]
6
7 df = spark.createDataFrame(order_data)
8
9 clv_df = df.groupBy("customer_id").agg(sum("order_amount").alias("total_spent"), count("order_id").alias("total_orders"))
10 print(clv_df.display())
```

▸ (2) Spark Jobs

- df: pyspark.sql.dataframe.DataFrame = [customer_id: long, order_amount: long ... 1 more field]
- clv_df: pyspark.sql.dataframe.DataFrame = [customer_id: long, total_spent: long ... 1 more field]

Table ▾ +

	customer_id ▲	total_spent ▲	total_orders ▲
1	1	150	2
2	2	200	1

↓ 2 rows | 2.18 seconds runtime

Identifying repeat customers with groupBy() and count()

Cmd 17

```
1 order_data = [  
2     {"order_id": 1, "customer_id": 1},  
3     {"order_id": 2, "customer_id": 2},  
4     {"order_id": 3, "customer_id": 1},  
5     {"order_id": 4, "customer_id": 3},  
6 ]  
7  
8 df = spark.createDataFrame(order_data)  
9  
10 repeat_customer_df = df.groupBy("customer_id").count()  
11 print(repeat_customer_df.display())
```

► (2) Spark Jobs

- `df: pyspark.sql.dataframe.DataFrame = [customer_id: long, order_id: long]`
- `repeat_customer_df: pyspark.sql.dataframe.DataFrame = [customer_id: long, count: long]`

Table ▾ +

	customer_id ▲	count ▲
1	1	2
2	2	1
3	3	1

Analyzing order placement frequency with groupBy() and count()

Cmd 19

```
1 order_data = [  
2     {"order_id": 1, "customer_id": 1, "order_date": "2023-10-01"},  
3     {"order_id": 2, "customer_id": 1, "order_date": "2023-10-05"},  
4     {"order_id": 3, "customer_id": 2, "order_date": "2023-10-10"},  
5 ]  
6  
7 df = spark.createDataFrame(order_data)  
8 df = df.withColumn("order_year", year(df["order_date"]))  
9  
10 order_frequency_df = df.groupBy("customer_id", "order_year").count()  
11 print(order_frequency_df.display())
```

► (2) Spark Jobs

- df: pyspark.sql.dataframe.DataFrame = [customer_id: long, order_date: string ... 2 more fields]
- order_frequency_df: pyspark.sql.dataframe.DataFrame = [customer_id: long, order_year: integer ... 1 more field]

Table ▾ +

	customer_id ▲	order_year ▲	count ▲
1	1	2023	2
2	2	2023	1

display commission of all salesmans

Cmd 21

```
1 # Create sample data
2 data = [
3     {"salesman_id": 1, "sales": 1000, "commission_rate": 0.1},
4     {"salesman_id": 2, "sales": 2000, "commission_rate": 0.15},
5     {"salesman_id": 3, "sales": 3000, "commission_rate": 0.2},
6 ]
7
8 # Create DataFrame from sample data
9 df = spark.createDataFrame(data=data)
10
11 # Calculate commission for each salesman
12 df = df.withColumn("commission", col("sales") * col("commission_rate"))
13
14 # Display commission for all salesmen
15 df.display()
```

▶ (3) Spark Jobs

▶ df: pyspark.sql.dataframe.DataFrame = [commission_rate: double, sales: long ... 2 more fields]

Table ▾ +

	commission_rate ▲	sales ▲	salesman_id ▲	commission ▲	
1	0.1	1000	1	100	
2	0.15	2000	2	300	
3	0.2	3000	3	600	

get unique salesman IDs from the order table without repeating salesman IDs

Cmd 23

```
1 data = [  
2     {"salesman_id": 1, "sales": 1000},  
3     {"salesman_id": 1, "sales": 2000},  
4     {"salesman_id": 2, "sales": 3000},  
5     {"salesman_id": 2, "sales": 4000},  
6     {"salesman_id": 3, "sales": 5000},  
7 ]  
8  
9 df = spark.createDataFrame(data=data)  
10  
11 unique_salesman_ids = df.select("salesman_id").distinct()  
12 unique_salesman_ids.display()
```

► (2) Spark Jobs

- df: pyspark.sql.dataframe.DataFrame = [sales: long, salesman_id: long]
- unique_salesman_ids: pyspark.sql.dataframe.DataFrame = [salesman_id: long]

Table ▾ +

	salesman_id ▲
1	1
2	2
3	3

get salesmans name and city from where city='bangalore'

Cmd 25

```

1 # Create sample data
2 data = [
3     {"salesman_id": 1, "salesman_name": "Reshwanth", "city": "Bangalore"},
4     {"salesman_id": 2, "salesman_name": "Vikranth", "city": "New York"},
5     {"salesman_id": 3, "salesman_name": "Raveendra", "city": "Bangalore"},
6 ]
7 # Define schema
8 schema = StructType([
9     StructField("salesman_id", IntegerType(), True),
10    StructField("salesman_name", StringType(), True),
11    StructField("city", StringType(), True),
12 ])
13 # Create DataFrame from sample data
14 df = spark.createDataFrame(data=data, schema=schema)
15 # Filter salesmen from Bangalore
16 bangalore_salesmen = df.filter(col("city") == "Bangalore")
17 # Select salesmen name and city
18 salesmen_info = bangalore_salesmen.select(["salesman_name", "city"])
19 # Display salesmen name and city
20 salesmen_info.display()

```

► (3) Spark Jobs

- df: pyspark.sql.dataframe.DataFrame = [salesman_id: integer, salesman_name: string ... 1 more field]
- bangalore_salesmen: pyspark.sql.dataframe.DataFrame = [salesman_id: integer, salesman_name: string ... 1 more field]
- salesmen_info: pyspark.sql.dataframe.DataFrame = [salesman_name: string, city: string]

Table ▾ +

	salesman_name ▲	city ▲
1	Reshwanth	Bangalore
2	Raveendra	Bangalore

get order number, date, qty, amount where deliverypersonid 200

Cmd 27

```
1 from pyspark.sql.types import *
2 from pyspark.sql.functions import *
3 # Define the schema for the sample data
4 schema = StructType([StructField("order_number", IntegerType(), True),
5     StructField("date", StringType(), True),
6     StructField("qty", IntegerType(), True),
7     StructField("amount", IntegerType(), True),
8     StructField("deliverypersonid", IntegerType(), True)])
9 # Create a sample DataFrame
10 data = [(1, "2023-01-01", 10, 100, 100),
11     (2, "2023-01-02", 5, 50, 200),
12     (3, "2023-01-03", 8, 80, 200),
13     (4, "2023-01-04", 12, 120, 300)]
14 # create dataframe and change date datatype to DATE using cast
15 df = spark.createDataFrame(data, schema=schema).withColumn("date", col("date").cast("date"))
16 # Filter the data for deliverypersonid = 200
17 filtered_data = df.filter(col("deliverypersonid") == 200)
18 # Select the required columns
19 result = filtered_data.select("order_number", "date", "qty", "amount")
20 # Show the result
21 result.show()
```

► (3) Spark Jobs

- df: pyspark.sql.dataframe.DataFrame = [order_number: integer, date: date ... 3 more fields]
- filtered_data: pyspark.sql.dataframe.DataFrame = [order_number: integer, date: date ... 3 more fields]
- result: pyspark.sql.dataframe.DataFrame = [order_number: integer, date: date ... 2 more fields]

order_number	date	qty	amount
2	2023-01-02	5	50
3	2023-01-03	8	80

How to Get First Day of the month and Last Date of the Month on any date

Cmd 38

```
1 from pyspark.sql.functions import *
2 # Sample DataFrame with a date column
3 data = [("2023-01-15",), ("2023-02-20",), ("2023-03-05",), ("2023-02-10",)]
4 columns = ["date"]
5 df = spark.createDataFrame(data, columns)
6 # Convert the string date column to a DateType
7 df = df.withColumn("date", df["date"].cast("date"))
8 # Calculate first and last dates of the month
9 df_result = df.withColumn("first_day_month", trunc(df["date"], "month")).withColumn("last_day_month", last_day(df["date"]))
10
11 # Show the result
12 df_result.show()
```

► (3) Spark Jobs

► df: pyspark.sql.dataframe.DataFrame = [date: date]

► df_result: pyspark.sql.dataframe.DataFrame = [date: date, first_day_month: date ... 1 more field]

date	first_day_month	last_day_month
2023-01-15	2023-01-01	2023-01-31
2023-02-20	2023-02-01	2023-02-28
2023-03-05	2023-03-01	2023-03-31
2023-02-10	2023-02-01	2023-02-28

To find the first visited customers by astore

Cmd 40

```

1 # Define the schema for the sample data
2 schema = StructType([StructField("customer_id", StringType(), True),
3   StructField("store_id", StringType(), True),
4   StructField("visit_date", StringType(), True)])
5 # Create a sample DataFrame
6 data = [("Cust1", "StoreA", "2023-01-01"),
7   ("Cust2", "StoreB", "2023-01-02"),
8   ("Cust1", "StoreA", "2023-01-03"),
9   ("Cust3", "StoreB", "2023-01-04"),
10  ("Cust2", "StoreA", "2023-01-05")]
11 df = spark.createDataFrame(data, schema=schema).withColumn("visit_date", to_date("visit_date"))
12 # Define a window specification to partition by customer_id and order by visit_date
13 windowSpec = Window.partitionBy("store_id").orderBy("visit_date")
14 # Add a column to identify the first time visited for each customer
15 df_result = df.withColumn("first_time_visited", min("visit_date").over(windowSpec)).filter(col("visit_date") == col("first_time_visited"))
16 # Select the relevant columns
17 result = df_result.select("customer_id", "store_id", "visit_date")
18 # Show the result
19 result.show()

```

► (2) Spark Jobs

- df: pyspark.sql.dataframe.DataFrame = [customer_id: string, store_id: string ... 1 more field]
- df_result: pyspark.sql.dataframe.DataFrame = [customer_id: string, store_id: string ... 2 more fields]
- result: pyspark.sql.dataframe.DataFrame = [customer_id: string, store_id: string ... 1 more field]

```

+-----+-----+-----+
|customer_id|store_id|visit_date|
+-----+-----+-----+
|      Cust1|  StoreA|2023-01-01|
|      Cust2|  StoreB|2023-01-02|
+-----+-----+-----+

```

To find the first-time visited customers in a store for a specific year using PySpark

Cmd 42

```
1 from pyspark.sql.functions import *
2 from pyspark.sql.types import *
3 # Define the schema for the sample data
4 schema = StructType([StructField("customer_id", StringType(), True),
5   StructField("store_id", StringType(), True),
6   StructField("visit_date", StringType(), True)])
7 # Create a sample DataFrame
8 data = [("Cust1", "StoreA", "2023-01-01"),
9   ("Cust2", "StoreB", "2021-01-02"),
10  ("Cust1", "StoreA", "2023-01-03"),
11  ("Cust3", "StoreB", "2020-01-04"),
12  ("Cust2", "StoreA", "2023-01-05"),
13  ("Cust1", "StoreA", "2022-12-30"), # Additional data for a different year
14 ]
15 df = spark.createDataFrame(data, schema=schema).withColumn("visit_date", to_date("visit_date"))
16 # Filter data for a specific year
17 target_year = 2023
18 df_filtered = df.filter(date_format(col("visit_date"), "yyyy") == target_year)
19 # Define a window specification to partition by customer_id and order by visit_date
20 windowSpec = Window.partitionBy("customer_id").orderBy("visit_date")
21 # Add a column to identify the first-time visited for each customer
22 df_result = df_filtered.withColumn("first_time_visited", min("visit_date").over(windowSpec)).filter(col("visit_date") == col("first_time_visited"))
23 # Select the relevant columns
24 result = df_result.select("customer_id", "store_id", "visit_date")
25 # Show the result
26 result.show()
```

▸ (2) Spark Jobs

- df: pyspark.sql.dataframe.DataFrame = [customer_id: string, store_id: string ... 1 more field]
- df_filtered: pyspark.sql.dataframe.DataFrame = [customer_id: string, store_id: string ... 1 more field]
- df_result: pyspark.sql.dataframe.DataFrame = [customer_id: string, store_id: string ... 2 more fields]
- result: pyspark.sql.dataframe.DataFrame = [customer_id: string, store_id: string ... 1 more field]

```
+-----+-----+-----+
|customer_id|store_id|visit_date|
+-----+-----+-----+
|    Cust1  |  StoreA|2023-01-01|
|    Cust2  |  StoreA|2023-01-05|
+-----+-----+-----+
```